

**Document Name:** Cyclic Executive Task Manager – System Design Document

**Document Reference:** CMPE 311-PROJECT-CE-SDD.1.1

**Date of Publication:** 9/29/2026

**LEAD ENGINEER:** Nyi Thu Kha, UMBC

## **STAKEHOLDERS:**

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

## **HIGH-LEVEL DESCRIPTION:**

An embedded system based on the ELEGOO Uno R3 that allows a user to set the blink intervals of two LEDs independently through the Arduino IDE serial monitor, using a round-robin cyclic executive task manager.

## **DESCRIPTION:**

The system uses an Arduino-compatible development board, two LEDs on digital pins 2 and 3, and a bare-metal, non-blocking software architecture organized as a round-robin cyclic executive with a function pointer table. The user enters which LED to change and the desired blink interval in milliseconds. Each LED then blinks at its own independent rate. Serial communication is asynchronous and hence the LED timing is unaffected.

## **RESULT SUMMARY:**

The system was designed, built, and tested. It successfully accepted serial commands, updated each LED interval independently. It also maintained constant blinking without any use of blocking delays or third-party multitasking libraries. All required tests, including those for the cyclic executive, were satisfied.

## REFERENCES

1. Nyi Thu Kha, Async LED Blink Controller — System Design Document, CMPE 311-ASYNC-LED-NYI-92626
2. [How to do Multitasking with the Arduino](#)
3. [The Arduino Handbook – Learn Microcontrollers for Embedded Systems](#)
4. sec04b-engineeringProcess\_pt1.pdf – Engineering Process Part 1
5. sec04b-engineeringProcess\_pt2.pdf – Engineering Process Part 2
6. CMPE310 asynchronous Arduino lab
7. Arduino IDE 2.3.3 or later documentation
8. [ATmega328P datasheet](#)

## DEFINITIONS

|                                    |                                                                                                                                                                                                             |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The User</b>                    | The person operating (not programming) the embedded system                                                                                                                                                  |
| <b>The System</b>                  | The embedded system being operated by <i>The User</i>                                                                                                                                                       |
| <b>The Customer</b>                | The person(s) paying for the embedded system being designed and built                                                                                                                                       |
| <b>The Developer</b>               | The person(s) designing and building the <i>System</i>                                                                                                                                                      |
| <b>The Evaluator</b>               | The person(s) that determine whether or not <i>The System</i> satisfies <i>The Customer-requirements</i>                                                                                                    |
| <b>The Customer Requirements</b>   | The requirements defined by <i>The Customer</i> as satisfying <i>The Contract</i>                                                                                                                           |
| <b>The HL-requirements</b>         | The System's high-level technical requirements derived from <i>The Customer-requirements</i>                                                                                                                |
| <b>The Requirements</b>            | context dependent; e.g. may refer to the <i>HL-requirements</i> or <i>Customer requirements</i>                                                                                                             |
| <b>The Educational-constraints</b> | The requirements imposed by the instructor unrelated to the embedded system that allow <i>The System</i> to be evaluated.                                                                                   |
| <b>The Company</b>                 | The organization on <i>The Customer</i> has contracted with to build <i>The System</i>                                                                                                                      |
| <b>The Contract</b>                | The business document that legally binds <i>The Company</i> to provide some service or product to <i>The Customer</i>                                                                                       |
| <b>serial-monitor</b>              | The serial port used by the Arduino IDE to communicate with <i>The User</i>                                                                                                                                 |
| <b>The Reference-platform</b>      | The configuration on of <i>The System</i> used by <i>The Developer</i> to test and validate <i>The System</i> . For this class, <i>The System</i> is the Arduino compatible ELEGOO Uno R3 development board |
| <b>Asynchronous</b>                | Serial communication and LED timing that do not block or wait on each other                                                                                                                                 |
| <b>Non-Blocking</b>                | Code that does not use <code>delay()</code> to pause execution                                                                                                                                              |

|                               |                                                                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>Millis()</b>               | Arduino function returning milliseconds since program start, used for timing                                                      |
| <b>Blink Interval</b>         | Full on/off period in milliseconds. On-time and off-time are each half the interval                                               |
| <b>Duty Cycle</b>             | Percentage of time the LED is on; this design used 50%                                                                            |
| <b>Cyclic Executive</b>       | A scheduling approach in which the main loop dispatches a fixed set of tasks in a repeating order                                 |
| <b>Round-Robin</b>            | A cyclic executive policy in which tasks are visited one at a time in a fixed order and the sequence wraps back to the first task |
| <b>Function Pointer Table</b> | An array of function pointers used to dispatch tasks by index                                                                     |
| <b>Task</b>                   | A function that performs one unit of work and returns promptly without blocking                                                   |
| <b>Dispatcher</b>             | The portion of <code>loop()</code> that selects and calls the next task                                                           |

## ACRONYMS AND ABBREVIATIONS

|                  |                                                                                                                  |
|------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Arduino</b>   | an Italian open-source hardware and software company; also refers to a development board created by that company |
| <b>arduino.h</b> | header for a library of convenience functions specific to the Arduino development platform                       |
| <b>AVR</b>       | A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology         |
| <b>avr.h</b>     | header including helper functions and definitions for using an AVR microcontroller                               |
| <b>ELEGOO</b>    | A Chinese company that develops and markets 3D printers and accessories                                          |
| <b>HL</b>        | High-level                                                                                                       |
| <b>IDE</b>       | Integrated Development Environment                                                                               |
| <b>gcc</b>       | Front end of the GNU Compiler Collection                                                                         |
| <b>GitHub</b>    | A widely used distributed SVC (Software Version Control) system                                                  |
| <b>LED</b>       | Light Emitting Diode                                                                                             |
| <b>RTOS</b>      | Real-Time Operating System                                                                                       |
| <b>ISR</b>       | Interrupt Service Routine                                                                                        |
| <b>SVC</b>       | Software Version Control                                                                                         |
| <b>UART</b>      | Universal Asynchronous Receiver/Trasmitter                                                                       |
| <b>CE</b>        | Cyclic Executive                                                                                                 |
| <b>ECN</b>       | Engineering Change Note                                                                                          |
| <b>ECO</b>       | Engineering Change Order                                                                                         |
| <b>ECR</b>       | Engineering Change Request                                                                                       |

## CONVENTIONS

- Must, shall or will** - your design must satisfy the requirement  
**May** - your design may satisfy the requirement but doesn't have to  
**Informative** - the intent of the following description is to make the requirement more understandable

All customer requirements are started with **C.#**

All high-level technical requirements are started with **HL.#**

All testing/validation requirements are started with **T.#**

## CUSTOMER REQUIREMENTS

- C1.** The User will be able to choose the blink rate of the LED with a button.
- C2.** The User will choose between 4 rates plus off.
- C3.** The blink rates must be set by an operator prior to the system being started.
- C4.** Permitted blink rates must be perceivable by the viewer.
- C5.** The minimum blink rate must be no slower than once every 5 seconds.
- C6.** The System must run upon an Arduino Uno R3 compatible board.

## PART I – HIGH-LEVEL TECHNICAL REQUIREMENTS DERIVED FROM C1-C6

- HL.1** The System must use an Arduino Uno R3 compatible development board.
- HL.2** The System must use at least one LED.
- HL.3** The System must use at least one momentary push-button.
- HL.4** The System must provide five user-selectable modes: OFF four blink rates.
- HL.5** The four blink rates must be nominally 0.2 Hz, 1 Hz, 2 Hz, and 5 Hz.
- HL.6** The User must select the mode using the pushbutton before normal operation begins.
- HL.7** The selected mode must be stored and used after the System starts.
- HL.8** The slowest blink rate must have a period no greater than 5.0 seconds.
- HL.9** Each blink rate must produce alternating ON/OFF states with a 50% duty cycle.
- HL.10** The System must not require a serial monitor or host computer for mode selection.
- HL.11** The System must run standalone once programmed and powered.

*Note: These Part I HLs are not used in Parts II and III.*

## PART II/III – HIGH-LEVEL TECHNICAL REQUIREMENTS GIVEN IN TABLE 3

**Table 1. High-Level Requirements**

|             |                                                                                                       |
|-------------|-------------------------------------------------------------------------------------------------------|
| <b>HL.1</b> | The System must use at least 2 LEDs.                                                                  |
| <b>HL.2</b> | The System must use a standard Arduino compatible development board, like the provided ELEGOO Uno R3. |
| <b>HL.3</b> | The System must communicate with The User via the Arduino IDE serial-monitor port.                    |
| <b>HL.4</b> | Any use of the serial-monitor in HL.3 must be asynchronous and not affect the blinking of the LEDs.   |
| <b>HL.5</b> | The User must set the blink interval of the LEDs in msec.                                             |
| <b>HL.6</b> | The blink rate of each LED must be constant unless changed by The User.                               |

## EDUCATIONAL CONSTRAINTS

**Table 2. Educational Constraints**

|                 |                                                                                                                                                                                                                      |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>EC.1</b>     | The Developer must use the version of gcc used by Arduino IDE 2.3.3 or later for the AVR ATmega328p to cross compile any program used by The System.                                                                 |
| <b>EC.2</b>     | The Developer must have any software used in The System hosted on a GitHub repository. This software does not need to include libraries, macros, or other code not developed by The Developer.                       |
| <b>EC.2.1</b>   | The Evaluator must be able to download from the repository any software, documentation, and other material needed for their evaluation, cross compile any software, and run that software on The Reference Platform. |
| <b>EC.3</b>     | LEDs must be connected to Arduino digital Pins 2 and 3.                                                                                                                                                              |
| <b>EC.4</b>     | The Evaluator must be able to successfully have The System perform the dialog in Table 5.                                                                                                                            |
| <b>EC.5</b>     | The Developer must do bare metal programming, i.e., they cannot use any 3rd party multi-tasking libraries or programs such as FreeRTOS.                                                                              |
| <b>EC.6</b>     | The Developer must implement a cyclic executive task manager based upon a function pointer table to dispatch the asynchronous tasks created in PROJECT-ASYNC.                                                        |
| <b>EC.6.1</b>   | The system testing and validation requirements must contain those defined in PROJECT-ASYNC.                                                                                                                          |
| <b>EC.6.1.1</b> | The testing and validation requirements must be extended to contain any additional tests necessary to properly evaluate and demonstrate the function of the modified system.                                         |
| <b>EC.6.2</b>   | The final report must be a new revision of the formal design document produced in PROJECT-ASYNC, and must note any changes made to the design.                                                                       |

## DEVELOPMENT PLATFORM

1. ELEGOO Uno R3 development board or compatible Arduino Uno R3
2. Arduino IDE 2.3.3 or better
3. gcc toolchain used by Arduino IDE for AVR ATmega328p
4.  $2 \times$  LEDs
5.  $2 \times 200\ \Omega \frac{3}{4}\ W$  resistors
6.  $1 \times 10\ k\Omega \frac{3}{4}\ W$  resistors
7. Breadboard and jumper wires
8. USB 2.0 female type B to female USB 2.0 type A cable
9. Laptop/computer running Arduino IDE
10. GitHub repository for source and documentation

## ADDITIONAL DESIGN CONSIDERATIONS

1. The hardware design is identical to that of PROJECT-ASYNC. See [1] SYSTEM DESIGN section.
2. The software design uses a round-robin cyclic executive with a function pointer table.
3. The software is organized into three non-blocking tasks: `taskSerial`, `taskLed1`, and `taskLed2`.
4. The dispatcher in `loop()` calls exactly one task per iteration and advances the task index modulo the number of tasks.
5. `millis()` is used for all LED timing.
6. Shared state is accessible to all tasks at file scope.
7. No `delay()` is used in the main loop.
8. No FreeRTOS or third-party multitasking library is used.
9. The serial monitor baud rate is 9600.

## SCHEMATIC DESCRIPTION

Identical to PROJECT-ASYNC. See [1] SCHEMATIC DESCRIPTION section.

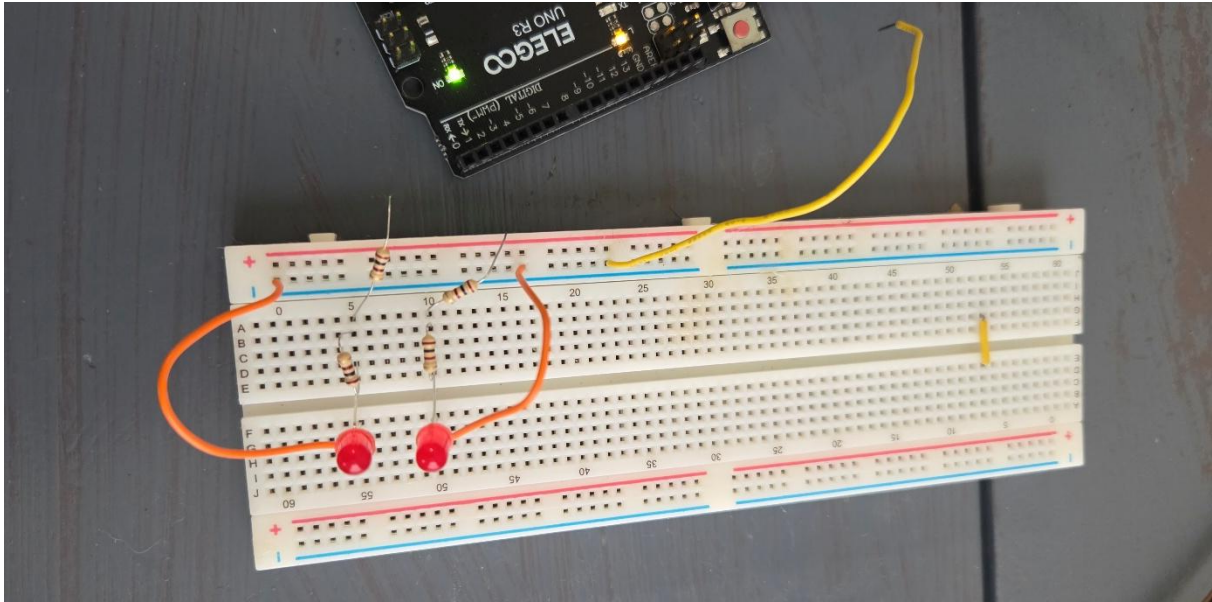


Figure 1 (Photo of physical Circuit). Note: 2 100  $\Omega$  resistors were used for 200  $\Omega$  resistors

## SYSTEM DESIGN

### 1. Overview

The hardware design is identical to that of PROJECT-ASYNC. See [1] ADDITIONAL DESIGN CONSIDERATIONS and SCHEMATIC DESCRIPTION sections.

The software design has changed. PROJECT-ASYNC used a single flat `loop()` that handled serial parsing and both LED toggles inline. PROJECT-CE restructures the same functionality into a round-robin cyclic executive driven by a function pointer table. The behavior visible to The User is unchanged.

### 2. Software Architecture

The software is organized into three non-blocking tasks:

| Task                      | Responsibility                                                      |
|---------------------------|---------------------------------------------------------------------|
| <code>taskSerial()</code> | Reads and parses serial input; updates the selected LED's interval. |
| <code>taskLed1()</code>   | Toggles LED1 when its half-interval has elapsed.                    |
| <code>taskLed2()</code>   | Toggles LED2 when its half-interval has elapsed.                    |

Each task returns promptly and never blocks. No task uses `delay()`, waits on I/O, or spins on a condition.

### 3. Cyclic Executive

The cyclic executive is implemented in `loop()`:

```
// Cyclic executive round-robin dispatch via the function pointer table
void loop() {
    static int currentTask = 0;
    taskTable[currentTask](); // task dispatch
    currentTask = (currentTask + 1) % NUM_TASKS; // for round-robin
}
```

On each pass through `loop()`, exactly one task is dispatched. The index advances by one, and then wraps back to zero after the last entry. This produces round-robin scheduling where every task is visited in a fixed order, once per cycle, with no priorities and no preemption.

#### 4. Function Pointer Table

```
// Function pointer table (cyclic executive)
typedef void (*TaskFunc)(void);
TaskFunc taskTable[] = {taskSerial, taskLed1, taskLed2};
const int NUM_TASKS = sizeof(taskTable) / sizeof(taskTable[0]); // Make it constant
```

The table has three properties that matter for this design:

1. **Uniform signature:** Every task takes no arguments and returns `void`.
2. **Compile-time size:** `NUM_TASKS` is computed with `sizeof`, so adding or removing a task automatically updates the modulus.
3. **Indirection:** `loop()` dispatches whichever function the current index points to.

#### 5. Task Design

`taskSerial()` drains the UART receive buffer and runs a two-state parser (`WAIT_LED`, `WAIT_INTERVAL`). It never blocks waiting for input.

`taskLed1()` and `taskLed2()` compare `millis()` against their own `lastToggle` timestamp. Because interval is the full period and each task toggles once per half-period, the resulting waveform is 50% duty cycle. LED1 and LED2 use separate intervals, state variables, and timestamps, so changing one does not affect the other.

#### 6. Timing Analysis

The scheduler guarantees tasks execute in a fixed order at a rate determined by `loop()`'s iteration speed. Because there are no tasks that block, the maximum interval between successive executions of any given task is bounded by the sum of all task execution times. Each task's execution time is on the order of microseconds, and since the observed worst-case cycle time is far below the smallest supported interval, it can be assumed that all timing requirements are met with wide margin.

#### 7. Changes from PROJECT-ASYNC



| Aspect                | PROJECT-ASYNC    | PROJECT-CE                     |
|-----------------------|------------------|--------------------------------|
| Main loop             | All logic inline | Round-robin dispatcher         |
| Task separation       | None             | taskSerial, taskLed1, taskLed2 |
| Dispatch mechanism    | Direct calls     | Function pointer table         |
| Scheduling            | N/A              | Round-robin                    |
| Hardware              | Unchanged        | Unchanged                      |
| User-visible behavior | Unchanged        | Unchanged                      |
| Testing requirements  | T.1-T.17         | T.1-T.17 + T.18-T.22           |

*See the accompanying ECN (ECN1.1) for the formal change record.*

## TESTING AND VALIDATION REQUIREMENTS

The Project 1 testing and validation requirements, T.0 through T.17, remain in effect. See [1] TESTING AND VALIDATION REQUIREMENTS section. In addition, PROJECT-CE adds the following:

**Table 3. Testing and Validation Requirements**

|             |                                                                                   |
|-------------|-----------------------------------------------------------------------------------|
| <b>T.18</b> | Verify the software implements a cyclic executive using a function pointer array. |
| <b>T.19</b> | Verify the main loop dispatches tasks in round-robin order.                       |
| <b>T.20</b> | Verify each task is non-blocking and returns to the dispatcher.                   |
| <b>T.21</b> | Verify that all tasks execute regularly without starving another.                 |
| <b>T.22</b> | Verify all Project 1 tests (T.0–T.17) still pass with the cyclic executive.       |

## TESTING RESULTS

**Table 4. Results of tests**

| Test performed | Results                                     |
|----------------|---------------------------------------------|
| T.0 – T.17     | Satisfied (see [1] TESTING RESULTS section) |
| T.18           | Satisfied                                   |
| T.19           | Satisfied                                   |
| T.20           | Satisfied                                   |
| T.21           | Satisfied                                   |
| T.22           | Satisfied                                   |

### Evidence:

```
What LED? (1 or 2)
What interval (in msec)?
What LED? (1 or 2)
What interval (in msec)?
What LED? (1 or 2)
```

Figure 2 (Serial Monitor Screenshot): *Inputs were in this order <2, 600, 1, 1600>*

[Video of independent blinking](#)

GitHub repository link: <https://github.com/nyithukha/CMPE311/tree/main/Project2>